

CLOUD PLATFORM RESEARCH

PROJECT: RARITONE BACKEND INFRASTRUCTURE RESEARCH

NANDINI BHIMINENI

INTRODUCTION:

The purpose of this research is to analyze cloud platforms and backend infrastructure technologies suitable for the Raritone virtual try-on application. The research focuses on scalable backend hosting, database management, image storage, authentication support, and AI integration capabilities.

2. Platforms Researched

Platform	Purpose
AWS	Cloud infrastructure and scalable services
Firebase	Authentication and storage services
Google Cloud	AI and cloud deployment support
Render	Backend hosting platform
Supabase	Backend-as-a-service platform
Cloudinary	Cloud-based media storage and CDN

3. AWS Research

Overview

AWS (Amazon Web Services) provides scalable cloud infrastructure solutions including compute, storage, database, and deployment services.

Services Explored

EC2

S3

Lambda

API Gateway

Advantages

Highly scalable

Reliable cloud infrastructure

Strong AI/ML support

4. Firebase Research

Overview

Firebase provides backend services including authentication, realtime database, cloud storage, and hosting.

Features Explored

Firebase Authentication

Firebase Storage

Cloud Firestore

Advantages

Easy integration

Fast setup

Realtime synchronization

5. Google Cloud Research

Overview

Google Cloud Platform provides cloud computing services suitable for AI model deployment and scalable backend applications.

Features Explored

AI Platform

Cloud Run

Cloud Storage

6. Render Research

Overview

Render is a cloud hosting platform used for backend deployment and scalable Node.js application hosting.

Features Explored

Backend deployment

Automatic deployments

Environment variable management

Advantages

Beginner friendly

Easy Node.js deployment

GitHub integration

7. Cloudinary Research

Overview

Cloudinary is a cloud-based media management platform used for image storage, optimization, and CDN delivery.

Features Explored

Image upload handling

CDN optimization

Secure image URLs

Cloud media storage

Advantages

Reduces backend storage load

Fast image delivery

Optimized media handling

8. Selected Technology Stack

Requirement	Selected Technology
Backend Framework	Node.js + Express
Database	MongoDB
Media Storage	Cloudinary
Authentication	JWT
Backend Hosting	Render

9. Conclusion

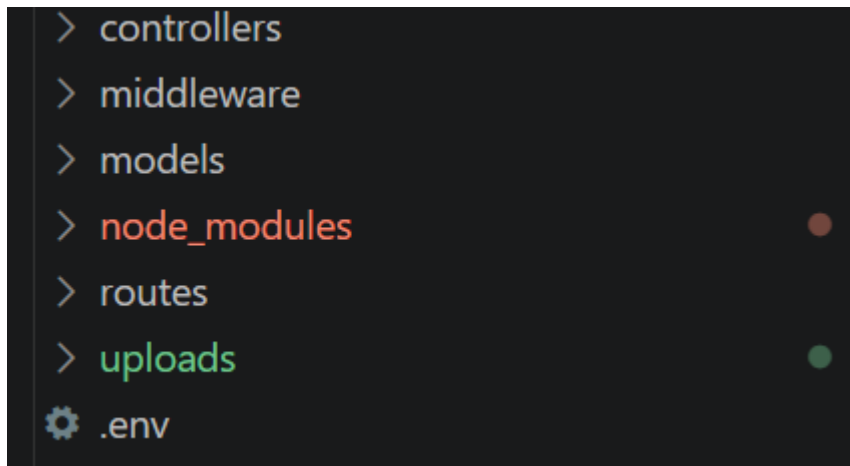
Based on the research, MongoDB, Cloudinary, Render, and JWT authentication were selected for the initial backend infrastructure setup of the Raritone application. The architecture supports scalability, cloud-based media handling, and future AI virtual try-on integration.

DATABASE SETUP AND SCREENSHOTS

```
[nodemon] restarting due to changes...  
[nodemon] starting `node server.js`  
◇ injected env (6) from .env // tip: ~ auth for agents [www.vestauth.com]  
Server running on 5000  
MongoDB Connected
```

```
▼ models  
  JS Image.js  
  JS product.js  
  JS Profile.js  
  JS TryOn.js  
  JS user.js  
  JS Wardrobe.js  
  JS Wishlist.js
```

```
.env × JS TryOn.js JS tryOnController.js JS db.js  
Raritone-Project-Backend > .env  
1  PORT=5000  
2  
3  MONGO_URI=mongodb://127.0.0.1:27017/raritoneDB  
4  
5  JWT_SECRET=raritoneSecretKey  
6  
7  CLOUDINARY_CLOUD_NAME=  
8  
9  CLOUDINARY_API_KEY=  
10  
11 CLOUDINARY_API_SECRET=
```



The backend infrastructure setup for the Raritone application was successfully initiated with MongoDB database integration, Cloudinary media storage configuration, JWT authentication structure, and scalable backend architecture planning. The system is designed to support secure, scalable, and AI-integrated virtual try-on functionalities for future development.

Progress Update

Today, the initial backend infrastructure setup and cloud integration research for the Raritone project were successfully completed.

Completed Work

- Researched cloud platforms including AWS, Firebase, Google Cloud, Render, Supabase, and Cloudinary
- Configured MongoDB database connection using Mongoose
- Implemented JWT authentication structure and protected route middleware
- Developed backend API structure for authentication, virtual try-on, wishlist, and wardrobe modules
- Integrated Cloudinary configuration and upload middleware for cloud-based image storage
- Created backend architecture planning for scalable AI virtual try-on support
- Implemented image upload handling and API testing using Postman
- Structured MongoDB collections for user-specific data management

Current Status

- Backend server running successfully

- MongoDB connection established successfully
- JWT authentication workflow implemented
- Cloudinary integration setup is done
- Upload API testing and debugging currently in progress
- Backend architecture and API planning completed

Technologies Used

- Node.js + Express
- MongoDB
- Mongoose
- JWT Authentication
- Cloudinary
- Multer
- Postman